

SplitStream — Portfolio Project PRD

Group Subscription Cost-Sharing Platform with Trust Scores

Version 1.0 | June 2026 | Portfolio Build

1. Project Purpose

For: Software engineering portfolio / job interviews

Goal: Build a full-stack web application that demonstrates:

- Real payment gateway integration (Stripe test mode)
- Complex business logic (trust scoring, automated billing)
- Clean architecture (MVC, REST API, state management)
- Production-ready deployment

Not a startup. No legal compliance, no real money, no merchant accounts.

2. Core Concept (1 Paragraph)

SplitStream lets users create “pools” for shared subscriptions (Netflix, Spotify, etc.). One user is the Host who sets up the pool. Others join as Contributors and pay their share via Stripe test payments. A Trust Score tracks payment reliability. Hosts are protected by automated retry logic and contributor replacement.

3. Tech Stack

Layer	Technology	Justification
Frontend	React 18 + Vite + Tailwind CSS	Fast dev, modern, great portfolio visuals
State	Redux Toolkit + React Query	Shows you know both client and server state
Backend	Node.js + Express.js	Standard, readable, interview-friendly
Database	MongoDB Atlas (M0 free tier)	NoSQL, Mongoose ODM, easy schema demo
Auth	JWT (access + refresh tokens)	Industry standard, shows security awareness
Payments	Stripe (test mode only)	Real API integration, instant setup, impressive
Scheduling	node-cron	Demonstrates async job handling
Deployment	Vercel (frontend) + Render (backend)	Free, auto-deploy, production URL for resume

4. Database Schema (3 Collections Only)

4.1 User

```
{
```

```

_id: ObjectId,
name: String,           // required, 2-50 chars
email: String,          // required, unique, lowercase
password: String,       // bcrypt hashed
trustScore: Number,     // default: 100, range: 0-100
stripeCustomerId: String, // from Stripe customer.create
role: String,           // enum: ['user', 'admin'], default: 'user'
createdAt: Date
}

```

4.2 Pool

```

{
  _id: ObjectId,
  hostId: ObjectId,     // ref: User
  serviceName: String,  // e.g., "Netflix", "Spotify"
  planTier: String,     // e.g., "Standard", "Premium"
  totalCost: Number,    // in cents (Stripe format)
  billingCycle: String,  // enum: ['monthly', 'annual']
  slots: Number,        // total slots (2-6 for demo)
  shareAmount: Number,   // totalCost / slots (auto-calculated)
  status: String,        // enum: ['draft', 'active', 'closed']
  members: [{
    userId: ObjectId,
    role: String,        // 'host' or 'contributor'
    joinedAt: Date,
    status: String       // 'active', 'defaulted', 'left'
  }],
  inviteCode: String,    // 8-char alphanumeric, auto-generated
  nextBillingDate: Date,
  createdAt: Date
}

```

4.3 Transaction

```

{
  _id: ObjectId,
  poolId: ObjectId,     // ref: Pool
  userId: ObjectId,     // ref: User (contributor who paid)
  type: String,          // enum: ['contributor_debit', 'host_payout', 'refund']
  amount: Number,        // in cents
  status: String,        // enum: ['pending', 'completed', 'failed']
  stripePaymentIntentId: String,
  retryCount: Number,    // default: 0, max: 3
  createdAt: Date
}

```

Why only 3 collections?

- Trust events are derived from Transaction data (no separate collection needed)
- Payment methods are handled by Stripe (no need to store card tokens)

- Keeps the project focused and interview-explainable in 5 minutes

5. API Endpoints (10 Total)

Auth (4)

Method	Endpoint	Description	Auth
POST	/api/auth/register	Create account	Public
POST	/api/auth/login	Get JWT tokens	Public
POST	/api/auth/refresh	Get new access token	Cookie (refresh)
POST	/api/auth/logout	Clear tokens	Cookie (refresh)

Pools (4)

Method	Endpoint	Description	Auth
POST	/api/pools	Create pool (Host only)	JWT
GET	/api/pools	List active pools	JWT
GET	/api/pools/:id	Get pool details	JWT
POST	/api/pools/:id/join	Join pool via invite code	JWT

Payments (2)

Method	Endpoint	Description	Auth
POST	/api/payments/intent	Create Stripe PaymentIntent	JWT
POST	/api/payments/confirm	Confirm payment + update status	JWT

Webhook (1)

Method	Endpoint	Description	Auth
POST	/webhooks/stripe	Handle Stripe events	Signature verify

Health (1)

Method	Endpoint	Description	Auth
GET	/api/health	Server status check	Public

6. Stripe Payment Flow

6.1 Test Mode Setup

1. Create Stripe account (free, instant)
2. Get test keys from Dashboard → Developers → API Keys
3. Use test card: 4242 4242 4242 4242 (always succeeds)
4. Use test card: 4000 0000 0000 0002 (always fails)

6.2 Contributor Payment Flow

```
1. Contributor clicks "Pay My Share" on pool page
2. Frontend calls POST /api/payments/intent
  Backend:
  - Creates Stripe PaymentIntent (amount = pool.shareAmount)
  - Saves Transaction record with status: 'pending'
  - Returns client_secret to frontend
3. Frontend uses Stripe.js to confirm card payment
4. Stripe redirects back or sends webhook
5. Webhook 'payment_intent.succeeded' arrives:
  - Find Transaction by stripePaymentIntentId
  - Update status to 'completed'
  - Update User trustScore: +2
  - Update Pool: check if all contributors paid
6. Webhook 'payment_intent.payment_failed' arrives:
  - Update Transaction status to 'failed'
  - Update User trustScore: -10
  - Schedule retry (node-cron, next day)
```

6.3 Host Payout (Mocked)

```
Trigger: All contributor transactions for cycle are 'completed'
1. Calculate: totalCollected - platformFee (5%)
2. Create Transaction: type='host_payout', status='completed' (mocked)
3. Update Pool: nextBillingDate += 1 month
4. In real app: this would be Stripe Connect transfer
```

6.4 Webhook Events Handled

Event	Action
payment_intent.succeeded	Mark transaction completed, +2 trust
payment_intent.payment_failed	Mark failed, -10 trust, schedule retry
charge.refunded	Mark transaction refunded

7. Trust Score System (Simplified)

7.1 Rules

Event	Points	Trigger
On-time payment	+2	Payment succeeds on first attempt
Retry success	+1	Payment succeeds on retry
Payment failed	-10	Payment fails after all retries
Pool default	-25	Contributor marked as defaulted

7.2 Impact

- **90-100:** Green badge, “Trusted”
- **70-89:** Yellow badge, “Good Standing”
- **50-69:** Orange badge, “Needs Improvement” — requires Host approval to join
- **< 50:** Red badge, “Restricted” — cannot join new pools

7.3 Implementation

- Calculate on-the-fly from Transaction history
- No separate TrustEvent collection
- Query: sum points from user’s transactions

8. Cron Jobs (2 Only)

Job	Schedule	What It Does
billing-reminder	Daily 9 AM	Find pools billing tomorrow, log to console (demo: show in UI)
retry-failed	Daily 10 AM	Find failed transactions with retryCount < 3, attempt retry

No complex scheduling. Just enough to show you understand cron jobs.

9. Frontend Pages (7 Pages)

9.1 Landing Page (1)

- Hero section with value prop
- How it works (3 steps)
- Demo CTA: “Try with Test Mode”

9.2 Auth Pages

- `/login` — Email + password form
- `/register` — Name + email + password form
- JWT stored in httpOnly cookie (show security awareness)

9.3 Dashboard (/dashboard)

- Your pools (as Host or Contributor)
- Upcoming payments
- Trust score badge with color
- Quick actions: Create Pool, Browse Pools

9.4 Browse Pools (/pools)

- List of active pools
- Filters: service name, price range, available slots
- Sort: price, host trust score
- Pool cards: service, price/slot, slots available, host name + trust score

9.5 Pool Detail (/pools/:id)

- Service info, plan, billing cycle
- Members list with trust scores
- “Join Pool” button (if slots available)
- “Pay My Share” button (if member, shows Stripe card element)
- Payment status for current cycle
- Host actions: close pool

9.6 Create Pool (/pools/create)

- Service name input
- Plan tier input
- Total cost input (in INR for display, stored in cents)
- Billing cycle toggle
- Slots slider (2-6)
- Live preview: “Each member pays X”
- Submit → generates invite code

9.7 Trust Profile (/profile)

- Current trust score with visual gauge
- Recent transactions with impact
- Tips to improve (static text)

10. State Management

Redux Toolkit Slices

Slice	State	Purpose
auth	user, isAuthenticated	Login status, user data
pool	pools, currentPool, filters	Pool list and detail
ui	toasts, modals, loading	UI feedback

React Query

Query Key	Endpoint	Purpose
pools	GET /api/pools	Browse page data
pool-detail	GET /api/pools/:id	Pool detail page
transactions	(derived)	Payment history

Why both? Shows you understand when to use each. Redux for global auth/UI. React Query for server state.

11. Security Measures

Measure	Implementation	Why It Matters
JWT in httpOnly cookies	<code>res.cookie('refreshToken', token, { httpOnly: true, secure: true, sameSite: 'strict' })</code>	XSS protection
bcrypt password hashing	<code>saltRounds: 12</code>	Industry standard
Input validation	<code>express-validator</code> on all endpoints	Prevents injection
Rate limiting	100 req/min global, 5 login/15min	DDoS/brute force protection
Stripe webhook signature	<code>stripe.webhooks.constructEvent()</code>	Prevents spoofing
CORS whitelist	<code>app.use(cors({ origin: ['https://your-app.vercel.app'] })))</code>	CSRF protection
Helmet.js	Security headers	Best practice

12. Testing Strategy (Portfolio Demo)

Type	Tool	What to Test	Coverage Goal
Unit	Jest	Trust score calculation, utility functions	80%
Integration	Supertest	All API endpoints	All 10 endpoints
E2E	Manual or Playwright	Full flow: register → create pool → join → pay	3 critical flows

No need for complex test suites. Just enough to mention in interviews.

13. Deployment

Service	What	URL
Vercel	React frontend	https://splitstream.vercel.app
Render	Express backend	https://splitstream-api.onrender.com
MongoDB Atlas	Database	mongodb+srv://...
Stripe	Test payments	https://dashboard.stripe.com/test

Environment Variables

```
# Backend (.env)
PORT=5000
MONGODB_URI=mongodb+srv://...
JWT_SECRET=your-256-bit-secret
JWT_REFRESH_SECRET=your-256-bit-refresh-secret
STRIPE_SECRET_KEY=sk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
CLIENT_URL=https://splitstream.vercel.app

# Frontend (.env)
VITE_API_URL=https://splitstream-api.onrender.com
VITE_STRIPE_PUBLISHABLE_KEY=pk_test_...
```

14. Demo Script (For Interviews)

14.1 2-Minute Pitch

“SplitStream is a group subscription sharing platform. I built it to demonstrate full-stack skills with real payment integration. Users create pools for services like Netflix, others join and pay via Stripe. A trust score system incentivizes reliable payments. It’s built with React, Node.js, MongoDB, and Stripe — all deployed and demo-ready.”

14.2 Live Demo Flow (3 minutes)

1. **Register** two test accounts (Host + Contributor)
2. **Host**: Create a Netflix pool, set 199/month, 4 slots
3. **Show**: Invite code generated, share amount calculated
4. **Contributor**: Browse pools, join via invite code
5. **Contributor**: Pay share using Stripe test card 4242...
6. **Show**: Webhook updates transaction, trust score +2
7. **Host**: Dashboard shows “1/3 paid”, payout pending
8. **Show**: Cron job logs (console or UI) for retry logic
9. **Show**: Trust score badge changes color

14.3 Technical Deep Dive (5 minutes)

- **Architecture**: MVC pattern, RESTful API, JWT auth

- **Payments:** Stripe PaymentIntent flow, webhook handling, idempotency
- **Trust Score:** Derived from transaction history, real-time calculation
- **Cron Jobs:** node-cron for billing reminders and retry logic
- **Security:** httpOnly cookies, bcrypt, rate limiting, Helmet
- **Deployment:** Vercel + Render, CI/CD via GitHub Actions

15. What to Skip (Intentionally)

Feature	Why Skip
Real payouts	Requires Stripe Connect + business entity
KYC upload	Legal complexity, not needed for demo
Insurance pool	Business logic overkill for portfolio
Admin panel	Not impressive to employers
Push notifications	Firebase setup for demo is noise
Email service	Console.log is enough
Redis	In-memory is fine for demo scale
PWA / Mobile app	Out of scope, focus on web
Real cron execution	Show logs, don' t need 24/7 scheduling
Unit tests for everything	80% coverage on core logic is enough

16. File Structure

```
splitstream/  
├─ apps/  
│   └─ web/                                # React frontend  
│       └─ src/  
│           ├── components/                # Reusable UI  
│           ├── pages/                    # 7 page components  
│           ├── store/                    # Redux slices  
│           ├── hooks/                   # Custom React hooks  
│           ├── services/                 # API client (Axios)  
│           └─ App.jsx  
│       ├── package.json  
│       └─ vite.config.js  
└─ api/                                    # Node.js backend  
    ├── src/  
    │   ├── config/                      # DB, Stripe, JWT config  
    │   ├── controllers/                 # Auth, Pool, Payment  
    │   ├── models/                     # User, Pool, Transaction  
    │   ├── routes/                     # API routes  
    │   ├── middleware/                  # Auth, validation, error  
    │   ├── services/                    # Business logic  
    │   ├── jobs/                        # Cron jobs (2 files)  
    │   └─ app.js  
    └─ tests/
```

```
|   |   | unit/
|   |   | integration/
|   |   | package.json
|   |   |
|   |   | .env.example
|   |   | README.md
```

17. Success Metrics (For Portfolio)

Metric	Target	How to Measure
Code quality	ESLint + Prettier pass	<code>npm run lint</code>
Test coverage	80% on core logic	<code>npm run test:coverage</code>
API response time	< 200ms	Render dashboard or logs
Demo flow	< 3 minutes	Practice timing
Deployed & live	24/7 uptime	Vercel + Render
GitHub commits	Clean history	Meaningful commit messages

18. Interview Talking Points

“Why MongoDB over PostgreSQL?”

“For this project, MongoDB’s flexible schema made sense for rapid iteration. Pools have dynamic member arrays, and trust scores are derived from transaction history. For a production fintech app, I’d use PostgreSQL with proper ACID transactions.”

“How do you handle race conditions when two users join the last slot?”

“I use MongoDB’s atomic operations. The join operation checks `members.length < slots` in the query condition, so only one succeeds. The other gets a ‘pool full’ error.”

“What about idempotency in payments?”

“Each `PaymentIntent` has a unique ID stored in the `Transaction` collection. Before creating a new charge, I check if a completed transaction exists for this user + pool + billing cycle.”

“How would you scale this?”

“Add Redis for session caching, move cron jobs to a separate worker process, use Stripe Connect for real payouts, add read replicas for MongoDB, and implement proper event-driven architecture with a message queue.”

19. Quick Start (Copy-Paste Ready)

Step 1: Clone & Install

```
git clone <your-repo>
```

```
cd splitstream

# Backend
cd apps/api
npm install

# Frontend
cd ../web
npm install
```

Step 2: Environment

```
# Copy .env.example to .env and fill in:
# - MONGODB_URI (from Atlas)
# - JWT_SECRET (random 32+ chars)
# - STRIPE_SECRET_KEY (from Stripe Dashboard)
# - STRIPE_WEBHOOK_SECRET (from Stripe CLI)
```

Step 3: Run Locally

```
# Terminal 1: Backend
cd apps/api
npm run dev

# Terminal 2: Frontend
cd apps/web
npm run dev

# Terminal 3: Stripe webhook forwarding
stripe listen --forward-to localhost:5000/webhooks/stripe
```

Step 4: Test

```
# Backend tests
cd apps/api
npm test

# Manual test: Use Stripe test card 4242 4242 4242 4242
```

Step 5: Deploy

```
# Push to GitHub main branch
# Vercel auto-deploys frontend
# Render auto-deploys backend
# Update CLIENT_URL in Render env vars
```

20. Checklist: Before You Code

- Stripe account created, test keys copied
- MongoDB Atlas free cluster created
- GitHub repo initialized with README
- Vercel + Render accounts created
- `.env.example` created with all keys
- Figma wireframe OR paper sketch of 3 key pages
- This PRD read and understood
- Decision: skip features in Section 15 confirmed

Ready to code when 8/8 are checked.

Built for portfolio. Interview-ready. Deployed in production.